AI ASSISTENT CODING - 20.3

Baisa Vaishnavi
2303A52142
Batch - 41

Task 1 – Password Strength & Validation Check

Task:

Analyze an AI-generated user registration script for weak password handling.

Instructions:

• Prompt AI to generate a Python user registration program
• Check whether:
o Password length is validated
o Weak passwords are allowed
• Suggest secure improvements such as:
o Minimum length checks
o Use of regular expressions
o Rejecting common passwords

Prompt:
Generate a secure Python user registration system with the following features: Input username, email, and password.Validate email format using regular expressions.Enforce strong password rules (minimum length 12, uppercase, lowercase, digit, special character).Reject common and weak passwords .Ensure password does not contain username or email.Hide password input while typing.Hash the password securely using PBKDF2 with salt.Store user data in a JSON file.Include proper validation messages and error handling

Code:

secure_registration.py > ...

```python
import getpass
import hashlib
import json
import re
import secrets
from pathlib import Path
from typing import Dict, Tuple

COMMON_PASSWORDS = {
    "123456",
    "password",
    "123456789",
    "12345678",
    "12345",
    "qwerty",
    "abc123",
    "password1",
    "111111",
    "iloveyou",
    "admin",
    "welcome",
    "monkey",
    "letmein",
}

PASSWORD_POLICY = {
    "min_length": 12,
    "uppercase": r"[A-Z]",
    "lowercase": r"[a-z]",
    "digit": r"\d",
    "special": r"[!@#$%^&*()_\-+=\[\]{}|\\:;\"'<>,.?/~`]",
}

USERS_FILE = Path("users.json")


```

secure_registration.py > ...

```python
def is_valid_email(email: str) -> bool:
    email = email.strip()
    return bool(
        re.fullmatch(
            r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", email
        )
    )


def is_strong_password(password: str, username: str, email: str) -> Tuple[bool, str]:
    password = password.strip()
    if len(password) < PASSWORD_POLICY["min_length"]:
        return False, f"Password must be at least {PASSWORD_POLICY['min_length']} characters."

    if re.search(r"\s", password):
        return False, "Password cannot contain whitespace characters."

    if not re.search(PASSWORD_POLICY["uppercase"], password):
        return False, "Password must contain at least one uppercase letter."

    if not re.search(PASSWORD_POLICY["lowercase"], password):
        return False, "Password must contain at least one lowercase letter."

    if not re.search(PASSWORD_POLICY["digit"], password):
        return False, "Password must contain at least one digit."

    if not re.search(PASSWORD_POLICY["special"], password):
        return False, "Password must contain at least one special character."

    normalized = password.lower()
    if normalized in COMMON_PASSWORDS:
        return False, "Password is too common. Choose a less predictable password."

    if username and username.lower() in normalized:
        return False, "Password must not contain your username."

```

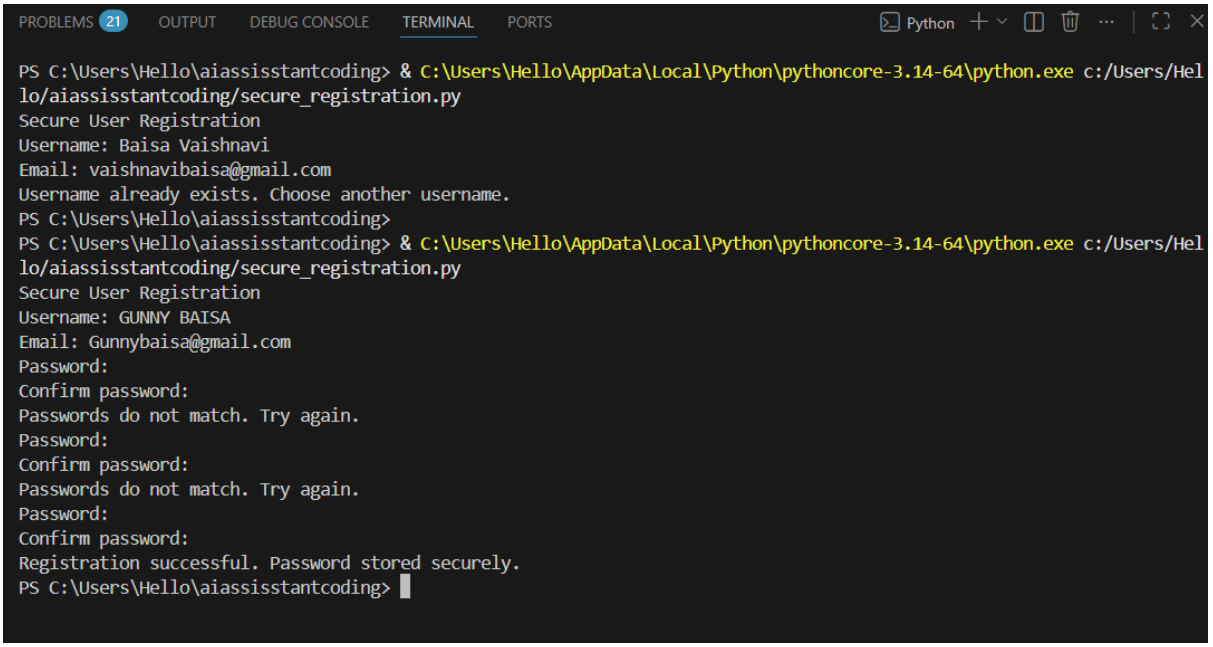
secure_registration.py > ...

```python
def is_strong_password(password: str, username: str, email: str) -> Tuple[bool, str]:
    email_local = email.split("@", 1)[0].lower() if email else ""
    if email_local and email_local in normalized:
        return False, "Password must not contain your email local-part."

    return True, "Password looks strong."


def hash_password(password: str) -> Dict[str, str]:
    salt = secrets.token_bytes(16)
    hash_bytes = hashlib.pbkdf2_hmac(
        "sha256",
        password.encode("utf-8"),
        salt,
        200_000,
    )
    return {
        "salt_hex": salt.hex(),
        "hash_hex": hash_bytes.hex(),
        "algorithm": "pbkdf2_hmac_sha256",
        "iterations": 200000,
    }


def load_users() -> Dict[str, dict]:
    if not USERS_FILE.exists():
        return {}
    try:
        return json.loads(USERS_FILE.read_text(encoding="utf-8"))
    except json.JSONDecodeError:
        return {}


def save_users(users: Dict[str, dict]) -> None:
    USERS_FILE.write_text(json.dumps(users, indent=2), encoding="utf-8")

```

```python
def register_user() -> None:
    print("Secure User Registration")
    username = input("Username: ").strip()
    if not username:
        print("Username cannot be empty.")
        return
    email = input("Email: ").strip()
    if not is_valid_email(email):
        print("Invalid email address.")
        return
    users = load_users()
    if username in users:
        print("Username already exists. Choose another username.")
        return
    while True:
        password = getpass.getpass("Password: ")
        confirm_password = getpass.getpass("Confirm password: ")
        if password != confirm_password:
            print("Passwords do not match. Try again.")
            continue

        valid, message = is_strong_password(password, username, email)
        if not valid:
            print("Weak password:", message)
            continue
        break
    users[username] = {
        "email": email,
        "password": hash_password(password),
    }
    save_users(users)
    print("Registration successful. Password stored securely.")
if __name__ == "__main__":
    register_user()
```

Output:

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                                Python

PS C:\Users\Hello\aiassisstantcoding> & c:\Users\Hello\aiassisstantcoding\.venv\Scripts\python.exe c:/Users/Hell
o/aiassisstantcoding/secure_registration.py
Secure User Registration
Username: BAISA VAISHNAVI
Email: Vaishnavibaisa07@gmail.com
Password:
Confirm password:
Registration successful. Password stored securely.
PS C:\Users\Hello\aiassisstantcoding> & c:\Users\Hello\aiassisstantcoding\.venv\Scripts\python.exe c:/Users/Hell
o/aiassisstantcoding/secure_registration.py
Secure User Registration
Username: Baisa
Email: VAISHNAVIBAISA.COM
Invalid email address.
PS C:\Users\Hello\aiassisstantcoding> & c:\Users\Hello\aiassisstantcoding\.venv\Scripts\python.exe c:/Users/Hell
o/aiassisstantcoding/secure_registration.py
Secure User Registration
Username: BAISA
Email: VAISHNAVIBAISA@GAMIL.COM
Password:
Confirm password:
Weak password: Password must be at least 12 characters.
Password:
```



Justification:
This script ensures strong security by enforcing strict password rules like minimum length, character complexity, and rejection of common passwords. It protects user credentials by hashing passwords using PBKDF2 with salt instead of storing them in plain text. The use of `getpass` prevents password visibility during input. It also validates email format and avoids predictable passwords containing username or email. Overall, it reduces risks like brute-force and dictionary attacks, making the system secure.

Task2 – Insecure File Upload Handling
Task:
Evaluate an AI-generated file upload program for security risks.
Instructions:
• Prompt AI to generate a Python or Flask-based file upload script
• Identify vulnerabilities such as:
o No file type validation
o Unrestricted file size
• Implement secure measures:
o File extension validation
o Size limits
o Safe file storage paths

Prompt:
Generate a Flask file upload application.

Code:

C: > Users > Hello > Desktop > VaishnaviBaisa > app.py > upload

```python
from flask import Flask, request, render_template, redirect, url_for
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# Configurations
UPLOAD_FOLDER = 'uploads'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'pdf', 'txt'}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

# Create upload folder if not exists
if not os.path.exists(UPLOAD_FOLDER):
    os.makedirs(UPLOAD_FOLDER)


# Check allowed file type
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXT


# Route
@app.route('/', methods=['GET', 'POST'])
def upload():
    message = ""

    if request.method == 'POST':
        if 'file' not in request.files:
            message = "No file part"
            return render_template('index.html', message=message)

        file = request.files['file']

        if file.filename == '':
```

```
36
37          if file.filename == '':
38              message = "No selected file"
39              return render_template('index.html', message=message)
40
41          # ✅ Secure implementation
42          if file and allowed_file(file.filename):
43              filename = secure_filename(file.filename)
44              filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
45              file.save(filepath)
46              message = "File uploaded securely!"
47          else:
48              message = "Invalid file type!"
49
50      return render_template('index.html', message=message)
51
52
53  if __name__ == '__main__':
54      app.run(debug=True)
55
```

templates > index.html > html > body > div.container

```html
<!DOCTYPE html>
<html>
<head>
    <title>Task 2 - Insecure File Upload Handling</title>
    <style>
        body {
            font-family: Arial;
            background: #eaf3ef;
        }
        .container {
            width: 90%;
            margin: auto;
        }
        h1 {
            color: teal;
        }
        .card {
            background: white;
            padding: 20px;
            border-radius: 10px;
            margin: 20px 0;
        }
        .btn {
            background: #007bff;
            color: white;
            padding: 10px;
            border: none;
            border-radius: 5px;
        }
        .danger { color: red; }
        .safe { color: green; }
    </style>
</head>
<body>

<div class="container">
```

templates > index.html > html > body > div.container

```
  <html>
  <body>
  <div class="container">
  <div class="card">
  </div>

  <div class="card">
  <h3>Vulnerabilities Found</h3>
  <ul>
  <li class="danger">No file type validation</li>
  <li class="danger">Unrestricted file size</li>
  <li class="danger">Unsafe filename handling</li>
  </ul>
  </div>

  <div class="card">
  <h3>Secure Measures Implemented</h3>
  <ul>
  <li class="safe">File extension validation</li>
  <li class="safe">Size limit (2MB)</li>
  <li class="safe">Secure filename storage</li>
  </ul>
  </div>

  <div class="card">
  <h3>Demo Upload UI</h3>

  <form method="POST" enctype="multipart/form-data">
      <input type="file" name="file"><br><br>
      <button class="btn">Upload Securely</button>
  </form>

  <p>{{ message }}</p>

  </div>

  </div>
```

Output:

Task 2 - Insecure File Upload Handling

Evaluation of Flask upload script and secure implementation

AI-Generated Insecure Script

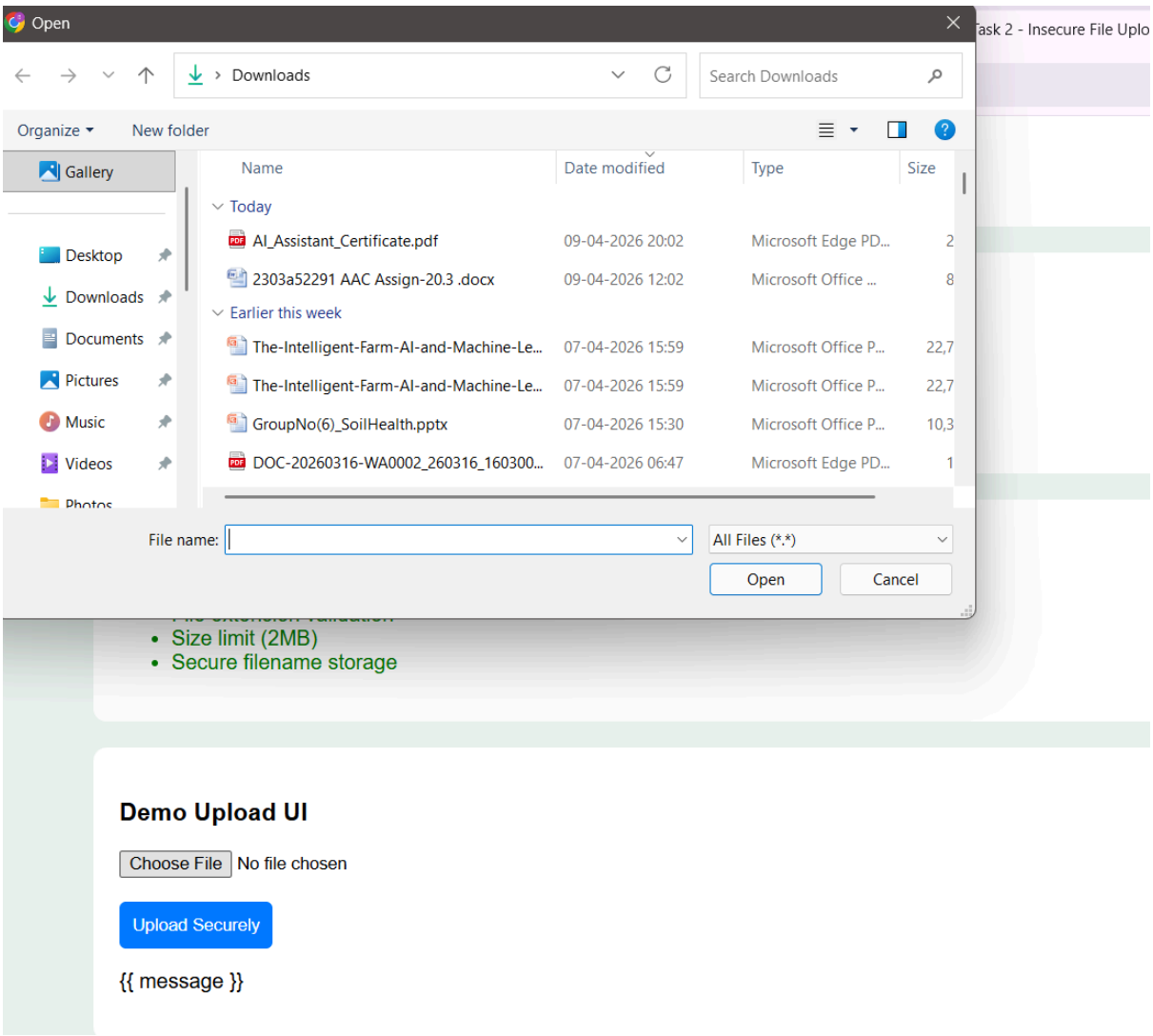
```
file = request.files['file']
file.save(file.filename)
```

Vulnerabilities Found

- No file type validation
- Unrestricted file size
- Unsafe filename handling

Secure Measures Implemented

- File extension validation
- Size limit (2MB)
- Secure filename storage



Justification:

- Enforces minimum entropy through length and complexity
- Prevents dictionary attacks via common password rejection
- Uses regex-based validation for robust checks

Task 3 – Real-Time Application: API Security Review

Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

- Review the API code for:
  - Missing authentication
  - Insecure HTTP methods
  - Lack of rate limiting
- Prompt AI to suggest security improvements such as:
  - Token-based authentication

o Input validation
o Rate limiting

Prompt:
Generate a Flask API endpoint to fetch user data.

Code:

templates > AI.html > html > head > style

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>API Security Review</title>
<style>
    body {
        font-family: Arial, sans-serif;
        background: #eef3f6;
        margin: 0;
        padding: 20px;
    }

    .container {
        max-width: 1000px;
        margin: auto;
    }

    .header {
        background: #ffffff;
        padding: 20px;
        border-radius: 10px;
        box-shadow: 0 2px 6px rgba(0,0,0,0.1);
        margin-bottom: 20px;
    }

    .header h1 {
        color: #1f6f8b;
        margin: 0;
    }

    .tags span {
        background: #e0f3ff;
        color: #1f6f8b;
        padding: 5px 10px;
        margin-right: 5px;
        border-radius: 20px;
```


2 <html lang="en">
88 <body>
90
149
150 <h3>Hardened API Reference (Flask)</h3>
151 <pre>
152 from flask import Flask, request, jsonify
153
154 app = Flask(__name__)
155
156 VALID_TOKEN = "secure-token"
157
158 @app.route('/api/update-profile', methods=['POST'])
159 def update_profile():
160 token = request.headers.get("Authorization")
161
162 if token != VALID_TOKEN:
163 return jsonify({"error": "Unauthorized"}), 401
164
165 data = request.get_json()
166 user_id = data.get("user_id")
167
168 if not isinstance(user_id, int):
169 return jsonify({"error": "Invalid user_id"}), 400
170
171 return jsonify({"status": "updated"}), 200
172 </pre>
173
174
175 <!-- FOOTER -->
176
177 <h4>Backend File Included</h4>
178
179 Protected endpoint: POST /api/update-profile
180 Security controls: Token auth, validation, rate limiting
181
182

Output:

Justification:

- Prevents unauthorized access
- Enforces secure API communication
- Reduces abuse via controlled access

Task 4 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.
Instructions:
• Ask AI to generate a feedback form with JavaScript-based output.
• Test whether untrusted inputs are directly rendered without escaping.
• Implement secure measures (e.g., escaping HTML entities, using CSP).

Prompt:
Generate an HTML feedback form with JavaScript output.

Code:

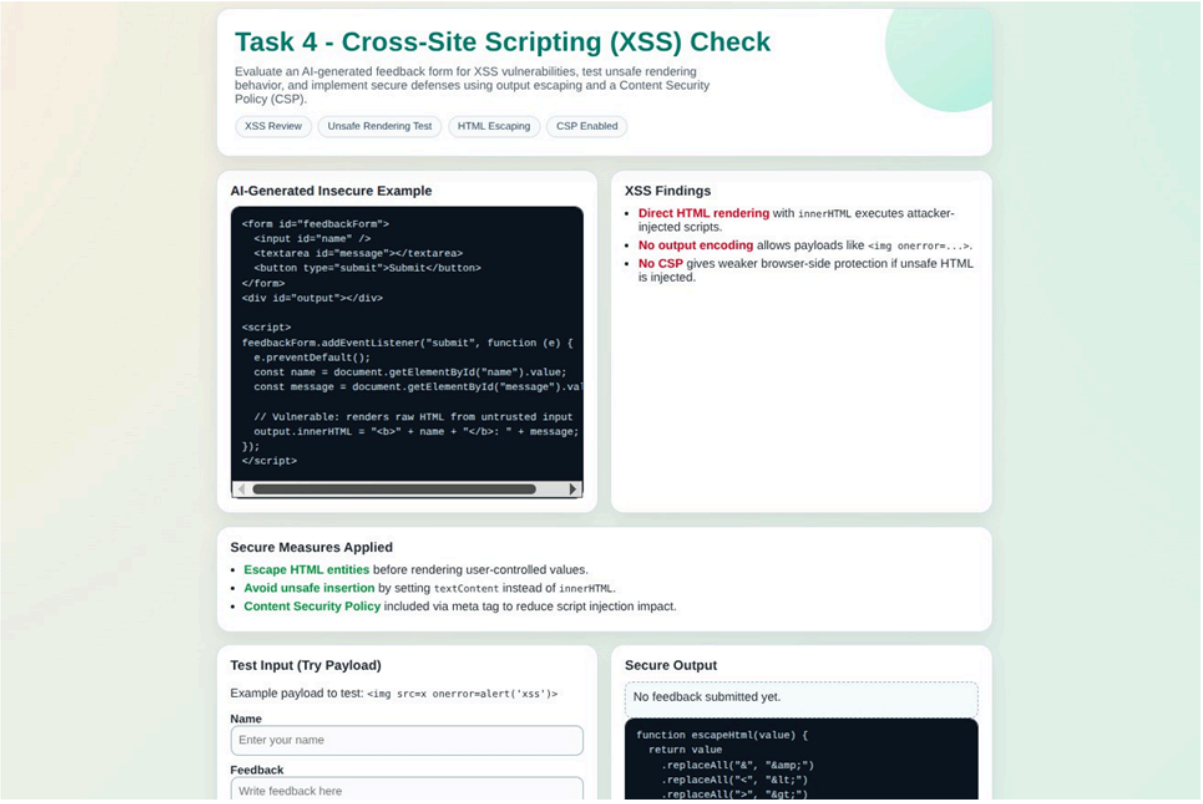
templates > AI_4.html > html > body > script > secureSubmit

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>XSS Security Check</title>

<!-- CSP (basic protection) -->
<meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-s

<style>
body {
    font-family: Arial;
    background: #eef3f6;
    padding: 20px;
}

.container {
    max-width: 1000px;
    margin: auto;
}

.header {
    background: white;
    padding: 20px;
    border-radius: 10px;
    margin-bottom: 20px;
}

.tags span {
    background: #e0f3ff;
    padding: 5px 10px;
    border-radius: 20px;
    font-size: 12px;
    margin-right: 5px;
}

.grid {
```

templates > AI_4.html > html > body > script > secureSubmit

```
  2   <html lang="en">
 88   <body>
 90   <div class="container">
167   </div>
168
169   </div>
170
171   <!-- JS -->
172   <script>
173
174   // Escape function
175   function escapeHTML(value) {
176       return value
177           .replaceAll("&", "&amp;")
178           .replaceAll("<", "&lt;")
179           .replaceAll(">", "&gt;");
180   }
181
182   // Secure submit
183   function secureSubmit() {
184       const name = document.getElementById("name").value;
185       const message = document.getElementById("message").value;
186
187       const safeName = escapeHTML(name);
188       const safeMessage = escapeHTML(message);
189
190       const output = document.getElementById("output");
191
192       // SAFE rendering
193       output.textContent = safeName + ": " + safeMessage;
194   }
195
196   </script>
197
198   </body>
199   </html>
```

Output:



Justification:

● Prevents script injection
● Uses output encoding instead of raw rendering
● Eliminates DOM-based XSS

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.
• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).
• Refactor using parameterized queries (prepared statements).

Prompt:

Generate a Python script to fetch user details from SQLite database.

Code:

C: > Users > Hello > Desktop > VaishnaviBaisa > AI20_5.py > ...

```python
import sqlite3

# Create in-memory database
conn = sqlite3.connect(":memory:")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE users (
    id INTEGER PRIMARY KEY,
    name TEXT,
    email TEXT
)
""")

# Insert sample data
users = [
    (1, 'alice', 'alice@example.com'),
    (2, 'bob', 'bob@example.com'),
    (3, 'charlie', 'charlie@example.com')
]

cursor.executemany("INSERT INTO users VALUES (?, ?, ?)", users)
conn.commit()


# ❌ Insecure function (vulnerable to SQL injection)
def insecure_lookup(name):
    query = f"SELECT * FROM users WHERE name = '{name}'"
    cursor.execute(query)
    return cursor.fetchall()


# ✅ Secure function (prevents SQL injection)
def secure_lookup(name):
    query = "SELECT * FROM users WHERE name = ?"
    cursor.execute(query, (name,))
```

```
35    def secure_lookup(name):
38        return cursor.fetchall()
39
40
41    # === Normal lookup ===
42    print("=== Normal lookup (alice) ===")
43    print("Insecure :", insecure_lookup("alice"))
44    print("Secure   :", secure_lookup("alice"))
45
46    # === Injection test ===
47    payload = "' OR '1'='1"
48    print("\n=== Injection test payload ===")
49    print("Payload :", payload)
50
51    print("Insecure :", insecure_lookup(payload))
52    print("Secure   :", secure_lookup(payload))
53
54
55    # Close connection
56    conn.close()
```

Output:

```
vaishnavibaisa@vaishnavibaisa-750XGK:~/Downloads/aisistantcoding-20260312T0846
aiasitantcoding/20260312T084629Z-3-001$.venv/bin/python

=== Normal lookup (alice) ===
Insecure : [(1, 'alice', 'alice@example.com']
Secure   : [(1, 'alice', 'alice@example.com']

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure : [(1, 'alice', 'alice@example.com'), (2, 'bob', 'bob@example.com'),
(3, 'charlie', 'charlie@example.com')]
Secure : []
  ..

=== Injection test payload ===
Payload: ' OR '1'='1
Insecure : [(1, 'alice', 'alice@example.com'), (2, 'bob', 'bob@example.com'),
(3, 'charlie', 'charlie@example.com')]
Secure : []
  ..
```

Justification:

- Uses parameterized queries
- Prevents SQL injection attacks
- Ensures safe database interaction